

AI-Based Smart Logistics & Accessibility Intelligence Platform — NER

Smart India Hackathon 2026 · Problem Statement SIH26002

Theme: Transportation & Logistics · Category: Software

Technical Documentation — Backend & Frontend

What was built, why, and how

Generated: 29 August 2026

Table of Contents

1. Executive Summary

This platform tackles the logistics and accessibility bottlenecks of India's eight North Eastern (NER) states — extreme cellular dead zones, monsoon landslides that sever single-road mountain corridors, and slow emergency response. It follows an "Edge-First, Spatially Aware" philosophy: devices keep working offline, spatial intelligence (PostGIS) drives geofencing and rerouting, and a multi-tier emergency fallback (2G SMS, BLE mesh) guarantees a distress signal can always leave the field.

What exists today: a production-shaped backend (Node.js + PostGIS + a Python AI microservice), two Flutter mobile apps (a Driver app and the Saptahara field-officer app), a React command-and-control dashboard, and a fleet simulator. All components were built, compiled, and verified running end-to-end (live smoke tests, an automated backend test suite, and on-device runs on an Android emulator).

Components at a glance

Component	Stack	Status
Backend API	Node.js / Express / TypeScript	Built & verified running
Spatial database	PostgreSQL + PostGIS	6+ tables, geofencing, dedup
AI microservice	Python / FastAPI / scikit-learn	Landslide risk model
Driver App	Flutter (offline-first)	Runs on emulator
Saptahara App	Flutter (field officer)	Integrated to backend
Command Dashboard	React / Vite / MapLibre	Live, real-time
Fleet Simulator	Node.js	Streams virtual trucks

2. System Architecture

The system is organised in three layers:

- **Edge layer** — the Flutter apps on drivers' / field officers' phones. They capture GPS, cache telemetry offline in SQLite, and trigger SOS over data, 2G SMS, or BLE mesh.
- **Core layer** — the Node/Express API backed by PostGIS and Redis, plus the FastAPI AI microservice. It ingests telemetry, runs spatial geofencing, computes hazard-aware routes, decodes emergencies, and broadcasts real-time events over Socket.IO.
- **Command layer** — the React dashboard for authorities (MDoNER evaluators' "control room"): live trucks on a map, hazard geofences turning red, and SOS alerts in real time.

Why this shape: the NER's defining constraint is unreliable connectivity, so the edge must be self-sufficient (offline-first) and the core must be resilient (fail-fast, graceful fallback). PostGIS was chosen because the core problems — is a vehicle inside a hazard zone, what is the safest route around it, what hazards are within N km — are fundamentally spatial queries.

Orchestration: everything runs via Docker Compose (PostGIS, Redis, the Node API, and the FastAPI service on one network). A single command brings the whole backend up.

3. Backend

3.1 What it is

A TypeScript Express API following a strict Controller → Service → Repository pattern, with Zod validation middleware and a centralised async error handler. It exposes a versioned REST API under /api/v1 plus a Socket.IO real-time channel.

3.2 Why these choices

- **Controller/Service/Repository** keeps HTTP concerns, business logic, and SQL cleanly separated and testable.
- **Zod validation** rejects malformed input at the edge of every route, returning consistent 422s.
- **Central error handler** maps Zod, PostgreSQL error codes, and typed AppErrors to clean HTTP responses (400/401/404/409/422/500).
- **Native DB deduplication** — a compound UNIQUE(vehicle_id, timestamp) with INSERT ... ON CONFLICT DO NOTHING means re-uploaded offline buffers can never create duplicates, without any application-side logic.

3.3 How it is built — key modules

Telemetry & resilient offline sync

POST /telemetry/batch bulk-inserts buffered edge points using native ON CONFLICT dedup and returns how many were genuinely new; it then runs a PostGIS geofence pass (ST_Contains) over the fresh points and pushes hazard:breach alerts. POST /telemetry/relay decodes a Base64 BLE-mesh frame.

Spatial hazards (PostGIS)

Hazard polygons are stored as GEOMETRY; queries use ST_DWithin (metre-accurate proximity), ST_Contains (geofence collision), and ST_Intersects (route-hazard crossing). GiST spatial indexes back every geometry column.

Multi-tier emergency

POST /emergency/sms decodes a Base64 distress payload (pipe or JSON frame → vehicle, lat, lng, message), persists an incident, anchors it to the audit ledger, and emits a real-time emergency:alert over Socket.IO. An unrecognised vehicle id is stored as NULL rather than dropping the distress signal.

Hazard-aware rerouting engine

A road network (road_nodes + road_edges) is held in PostGIS. For a trip, one SQL query flags every edge whose LineString ST_Intersects an active hazard polygon; a Dijkstra shortest-path then runs twice — once on raw distance, once with hazard-crossing edges penalised ×1000 — so the safest route detours around live geofences. The response classifies the result as clear, rerouted, or blocked_no_alternative and returns both the recommended and direct routes as GeoJSON.

JWT authentication

POST /auth/register (bcrypt-hashed) and /auth/login issue JWTs; a requireAuth middleware guards operator routes (trips, ledger append). Edge/gateway routes (telemetry, emergency, fleet, public hazard reads) stay open by design, as they would use device/service credentials in production.

Tamper-evident audit ledger (blockchain-inspired)

Each ledger entry stores the SHA-256 hash of its own fields plus the previous entry's hash (a hash chain from a genesis of 64 zeros). Emergencies and field reports are auto-anchored to it. GET /ledger/verify recomputes the whole chain and returns valid / the first broken sequence — so any retroactive edit to an old record is provably detectable. This is blockchain's cryptographic tamper-evidence, but single-writer (not a distributed/consensus blockchain).

3.4 API surface

Endpoint	Purpose
GET /health	Liveness
POST /auth/register · /login · GET /auth/me	JWT auth
POST /telemetry/batch · /relay	Offline bulk sync · BLE relay
GET/POST /hazards, /hazards/nearby, /contains, /risk	Geofencing & risk
POST /emergency/sms	2G SMS distress webhook
GET /fleet	Vehicles + last position
POST/GET/PATCH /trips, /trips/:id/track	Trip management (JWT)
GET/POST /ledger, /ledger/verify	Tamper-evident ledger
GET /geo/context · /risk · /layers	ISRO satellite feeds
POST /routing/route	Hazard-aware rerouting
POST/GET /reports	Field reports (ledger-anchored)
POST /predict (AI :9000)	Landslide risk model

3.5 AI microservice

A FastAPI service exposes POST /predict (inputs: latitude, longitude, rainfall_mm, soil_moisture, slope). It blends a physically-motivated monsoon-landslide heuristic with a scikit-learn logistic-regression model trained on synthetic, physically-labelled samples at start-up, and returns a 0–1 risk_score, a recommended_action (PROCEED / CAUTION / REROUTE / HALT), and a factor breakdown. It is honestly a heuristic + synthetic model, not trained on real ground-truth data.

4. ISRO Satellite Integration

4.1 Integration status (important)

Layer	ISRO integrated?	Detail
Backend (node-api)	YES — code + endpoints	Bhuvan, MOSDAC, CartoDEM clients; /geo endpoints. Returns a modelled fallback until real tokens are set.
Command Dashboard (React)	YES — surfaced	"Satellite Risk" card calls GET /geo/risk and shows the fused result + source tags.
Saptahara app (Flutter)	NO — not yet surfaced	Does not call the /geo endpoints. A drop-in addition.
Driver app (Flutter)	NO	Uses /hazards/risk (AI only), not the ISRO feeds.

In short: ISRO is integrated in the BACKEND (with a graceful modelled fallback) and surfaced on the DASHBOARD. It is not yet wired into the two mobile apps.

4.2 What was built

- **Bhuvan client** — WMS GetFeatureInfo against Bhuvan (NRSC) for landslide-susceptibility class at a point.
- **MOSDAC client** — satellite-derived rainfall + soil-moisture for a coordinate.
- **CartoDEM client** — elevation from the Cartosat-1 DEM (via Bhuvan) at a 5-point stencil, with slope derived from the elevation differences.
- **geo.service** — fuses all three feeds and drives the FastAPI landslide model with the real satellite-derived inputs, then blends with Bhuvan susceptibility for a final rerouting decision.

Every field is tagged with its true source: live (mosdac / cartodem / bhuvan_wms) or modelled. This is deliberate honesty — live NRSC/SAC data requires ISRO-issued tokens, so without them the service returns a physically-plausible NER monsoon/terrain fallback rather than pretending.

4.3 How to use the real ISRO API

Two steps — get tokens, then set environment variables. No code change is required; the clients switch from "modelled" to "live" automatically.

Step 1 — Register and obtain tokens

- **Bhuvan (NRSC/ISRO)**: register at bhuvan.nrsc.gov.in and request an API access token / API key for the WMS/thematic services (landslide susceptibility, CartoDEM terrain layers).
- **MOSDAC (SAC/ISRO)**: register at mosdac.gov.in and request access to the point-data / rainfall products.
- **CartoDEM**: distributed via Bhuvan — the same Bhuvan credentials typically cover the DEM/elevation service.

Step 2 — Configure the backend

Set these variables (in docker-compose.yml under the node-api service, or in node-api/.env), then restart the container:

```
BHUVAN_TOKEN=<your token>
MOSDAC_TOKEN=<your token>
CARTODEM_TOKEN=<your token>
# optional overrides:
BHUVAN_WMS_URL, BHUVAN_LANDSLIDE_LAYER, MOSDAC_BASE_URL, CARTODEM_URL
```

Restart: `docker compose up -d --build node-api`. Verify with `GET /api/v1/geo/context?lat=27.5&lng=92.1` — the "sources" fields should flip from "modelled" to "mosdac"/"cartodem"/"bhuvan_wms".

Step 3 (optional) — Surface on the dashboard / apps

The dashboard already shows the fused result. To use live tiles, `GET /geo/layers` returns the Bhuvan/CartoDEM WMS layer descriptors that a map (MapLibre or flutter_map) can add directly.

5. Frontend

5.1 Driver App (Flutter, edge layer)

What: an offline-first driver app — continuous GPS capture, telemetry sent when online and cached to SQLite when offline, background sync (WorkManager), and a multi-tier SOS (HTTP → Base64 2G-SMS → BLE mesh).

Why: the NER's cellular dead zones mean the app must lose nothing offline and must always be able to raise an SOS.

How: a clean service split — LocationService (send/queue), SyncService (batched drain with native dedup, foreground + background isolate), EmergencyService (the fallback ladder). The map is flutter_map / OpenStreetMap. Verified running on an Android 16 emulator: map, network-status pill, safety banner, and the SOS button all render, backed by 3 passing widget tests.

5.2 Saptahara App (Flutter, field officer)

What: a clean-architecture field-officer app (routes, alerts, offline reports, verification, SOS). It arrived as a well-built but fully mock-only app (no networking) and was integrated to the live backend.

Why: its domain model mapped almost one-to-one to the backend, and it was designed with a "swap Mock for Api" seam — the correct place to connect real data.

How — the integration:

Feature	Now backed by
Routes (map + "274 km rerouted")	POST /routing/route (reroute engine)
Current Alerts	GET /hazards + live Socket.IO
Geofences	GET /hazards polygons
Report sync	POST /reports (real upload)
Verification receipt	the ledger hash returned at sync
SEND SOS	POST /emergency/sms

Further enhancements, all verified on the emulator:

- Mock canvas replaced with a real flutter_map / OpenStreetMap tile view (routes, geofences, hazard markers plotted on real NER geography).
- Live socket indicator — the LIVE / RECONNECTING pill reflects the real Socket.IO connection (proven by stopping and restarting the backend).
- Socket-alert toasts — a new emergency:alert / hazard:breach surfaces an app-wide, severity-coloured toast.
- Toast deep-link — tapping a toast jumps to the Home map and flies the camera to the alert with a highlight ring.

5.3 Command Dashboard (React / Vite)

What: the authorities' control room — a MapLibre map showing live trucks, hazard geofences, SOS alerts, a satellite-risk panel, a hash-chain audit-ledger panel, and the recommended route.

Why: MDoNER evaluators want to see the central picture, not just a driver's phone.

How: socket.io-client streams fleet:update / hazard:breach / emergency:alert; the map draws truck markers, hazard polygons (turning red on breach), pulsing SOS pings, and the route as a green (recommended) vs red-dashed (direct) polyline that re-draws when a hazard activates. Clicking the map queries GET /geo/risk (the ISRO fused risk). All verified live.

6. Simulator & Verification

6.1 Fleet simulator

A Node.js script streams four virtual trucks along the Guwahati → Tawang highway: it seeds the vehicles into PostGIS, plants a Sela Pass hazard geofence, streams telemetry batches (driving live fleet movement + geofence breaches), and fires a scripted SOS mid-route — so judges can watch the whole pipeline without physically driving with a phone.

6.2 What was verified

- Backend: docker compose up, all containers healthy; every endpoint smoke-tested live (dedup, ST_DWithin/ST_Contains geofencing, AI risk, SMS decode, error paths).
- Automated tests: 42 passing backend tests (auth/bcrypt/JWT, emergency decode, requireAuth, Dijkstra/reroute, hash-chain tamper-detection, and supertest API tests).
- Rerouting: clear → rerouted transition proven (direct route through Sela Pass vs safe detour).
- Driver app & Saptahara: built and run on the Android emulator with live backend data.
- Dashboard: live trucks, geofence-red-on-breach, SOS pop-ups, satellite risk, and route rendering all confirmed in the browser.

7. What Remains To Be Added

7.1 Backend

- **Real ISRO tokens** — flip Bhuvan/MOSDAC/CartoDEM from modelled fallback to live (section 4.3).
- **Real outbound SMS gateway** — /emergency/sms decodes distress but does not yet send an alert out to responders.
- **Trip ↔ telemetry linking** — telemetry does not yet carry an active trip_id, so /trips/:id/track is currently empty.
- **Security hardening** — WebSocket (Socket.IO) auth, rate limiting/throttling, refresh tokens + revocation, and secrets management (JWT secret and DB password are currently in compose).
- **Admin CRUD** — endpoints for users and vehicles; pagination on list endpoints.
- **Migrations tooling** — replace manual ALTERs with a migration framework (e.g. node-pg-migrate).
- **Scale & ops** — TimescaleDB / partitioning + a queue (Kafka) for fleet-scale telemetry; structured logging, metrics, and a CI/CD pipeline running the tests.
- **Real routing graph** — the Dijkstra reroute uses a curated NER corridor graph; a city-scale version would load a full OpenStreetMap network via osm2pgrouting / pgRouting.

7.2 Frontend

- **Saptahara: surface ISRO satellite risk** (a /geo/risk card), dedupe repeated alert entries, extend toasts/deep-link to hazard:breach, wire real GPS (geolocator) instead of the fixed demo coordinate, and persist offline reports to disk (currently in-memory).
- **Auth screens** — a login flow using /auth so operator actions carry a real JWT.
- **Push notifications** — background alerts when the app is closed.
- **Localization & accessibility** — NER languages and accessible layouts (the platform's "Accessibility" mandate).
- **Dashboard: authentication** and a few more operational KPIs / historical playback.

8. How To Run

8.1 Backend (all four services)

Start Docker Desktop, then from the project root:

```
docker compose up -d --build
```

Verify: `curl http://localhost:8080/api/v1/health`. Run tests: `cd node-api && npm test`.

Note: on this machine the Docker CLI is a per-user install not on PATH. If "docker" is not recognised, prepend %LOCALAPPDATA%\Programs\DockerDesktop\resources\bin to PATH.

8.2 Full live demo

In separate terminals: `cd dashboard && npm run dev` and `cd simulator && node simulate.js`. Open `http://localhost:5173` — trucks move, the SOS fires, geofences turn red, the ledger chains, and satellite risk is one click away.

8.3 Mobile apps (emulator)

Flutter lives on `D:\flutter`. Launch the emulator, then from an app folder (driver-app or

saptahara_frontend/saptahara): `flutter run -d emulator-5554`

`--dart-define=API_BASE_URL=http://10.0.2.2:8080`.

10.0.2.2 is the Android emulator's alias for the host's localhost; a physical device uses the host LAN IP.